MESSI: A Nextflow pipeline for benchmarking multiomics integration methods for disease classification

Chunqing Tony Liang Tajveer Grewal Asees Singh Amrit Singh

Background

Technological advances have allowed for the profiling of different molecular measurements (e.g., genes, proteins, metabolites) from the same biological samples; termed multiomics (Hasin *et al.*, 2017; Subramanian *et al.*, 2020).

The majority of multiomics data are obtained for samples from individual patients (bulk multiomics data) and more recently with single-cell and spatial resolution.

Many methods have been developed to jointly analyze multiomics data to identify common patterns between datasets, and to identify biomarkers of disease (Vandereyken *et al.*, 2023).

This integrative approach of multiomics data may strengthen the understanding of the molecular dynamics underlying the biological processes of diseases and may lead to novel strategies for early detection, prevention, and treatment of human diseases (Sun & Hu, 2016).

Researchers face difficulties in choosing the right method due to the varying nature of analytical tasks such as clustering analysis, factor analysis, or cancer type prediction.

Hence, numerous reviews have benchmarked various integrative methods (Bersanelli *et al.*, 2016; Cantini *et al.*, 2021; Hasin *et al.*, 2017; Huang *et al.*, 2017; Li *et al.*, 2018; Li *et al.*, 2022; Luecken *et al.*, 2022; Pucher *et al.*, 2019; Richardson *et al.*, 2016; Yu & Zeng, 2018; Zeng & Lumley, 2018). However, these studies often focus on methods for specific tasks or data types and may lack reproducibility.

To address this, we propose a new automated framework to systematically benchmark and analyze multiomics methods with varying data types. This framework ensures full reproducibility and provides standardized input for any analytical task, with the capability to run on any computing platform.

Results

MESSI pipeline

We introduce here the MESSI pipeline, *Multiple Experiments with SyStematic Interrogation* (Fig 1, and additional supplementary), which comprises of 4 main steps: 1) Prepare data, 2) Data splitting, 3) Cross validation, 4) Feature selection.

MESSI is implemented with Nextflow (Di Tommaso *et al.*, 2017), a domain specific language (DSL) primarily used by bioinformaticians. Compared to traditional workflow management systems like Snakemake (Köster & Rahmann, 2012) or GNU make (Stallman *et al.*, 1988), Nextflow breaks down complex workflows into modular components, and connect them with channels which determines the flow of pipeline. This feature allows us to customize each module, extending the pipeline to not only benchmark purposes but also multi-purpose usage. Additionally, This modular design enables rapid, efficient and reproducible way to test and maintain codebases.

Moreover, the core feature of Nextflow of using containerization for modules like Docker (Docker *et al.*, 2020) and Singularity (Kurtzer *et al.*, 2017) solves the reproducibility issue in replicating papers. Each time the pipeline is executed, containers are created for each process defined in the workflows that constitute the pipeline. These containers follow same sets of operating system configurations, software versions of the desired computation performed. Hence, it fulfills reproducibility and portability of our results as each time we run these independently on its own encapsulated environment. Furthermore, the independence between each process leads nature of parallelizable

computations. This characteristic of the pipeline along with the resumability of re-executing interrupted or failed processes make debugging or changing computational settings for certain methods only atomic and simple. Ultimately, reducing time complexity of dealing with large datasets or long runtime computations, compared to standard way of sequentially executing complex scripts.

In terms of interoperability, Nextflow is capable to run the pipeline on various computing platforms including and not limited to our own personal computer, mainstream high-performance computing clusters (HPC) like SLURM, PBS or popular cloud platforms like AWS and Google cloud without modifying code logics. This is enabled from resource and parameters configurations, making user to only worry about configurations to carry out and the code logic itself.

Standardized data input in both R and Python

To ensure the standardized input, we incorporated the MultiAssayExperiment (Ramos *et al.*, 2017) package of the Bioconductor community (Gentleman *et al.*, 2004) for R (R Core Team *et al.*, 2013) and MuData (Bredikhin *et al.*, 2022) for Python (Sanner *et al.*, 1999). These tools are easily transferable, meaning users familiar with one can have the pipeline automatically convert to the other.

Currently, we are benchmarking existing methods like DIABLO (Singh *et al.*, 2019), Cooperative Learning (Ding *et al.*, 2022), MOGONET (Wang *et al.*, 2021), RGCCA (Girka *et al.*, 2023), MOFA (Argelaguet *et al.*, 2018, 2020) . The current pipeline focuses on testing this selection of integrative methods against a series of real multiomics datasets, particularly for classification tasks.

The results are compared using metrics like balanced classification error, F1 score, and biological enrichment results.

Through MESSI, we could systematically identify the top-performing methods with respect to classification performance and identifying molecular drivers of disease. Furthermore, this could extend to regression tasks or clustering tasks. This project can greatly benefit the community, either by developing new methods to assess their strength or finding a method that best suits custom multiomics data.

Prepare data

- Mention how data expect to be in tar gz containing the MAE and MuData
- These data goes to a preprocess step of filtering
- And cleaning up "columns" or metadata of it
- Describe some parameters used here, that could be controlled through nextflow config
- So from here, we come to commonly used and cleaned mae and mudata for respective language method

Data Splitting

- This splitting is because not all methods carry a built in cv
- Moreover, this make sure all method start with same data setup, since cv in each method might vary due to random seed problem
- Stratified to make sure of handling class imbalance, possible to go into discussion

Cross Validation

- Have a more detail flow of how here works, given could parallel per method, data, fold
- Show some plot from each method or data
- Then this makes each method as one workflow, where each can contain different number of modules, but requirement is just produce some kind of csv as end result that follows some format
- each workflow works in parallel and group by languages

Feature Selection

- This is an additional workflow that go directly after prepare data rather than splitting
- also makes it parallel by method
- then just talk how collect stuff in common format and produce series of metrics file that could be visualize downstream

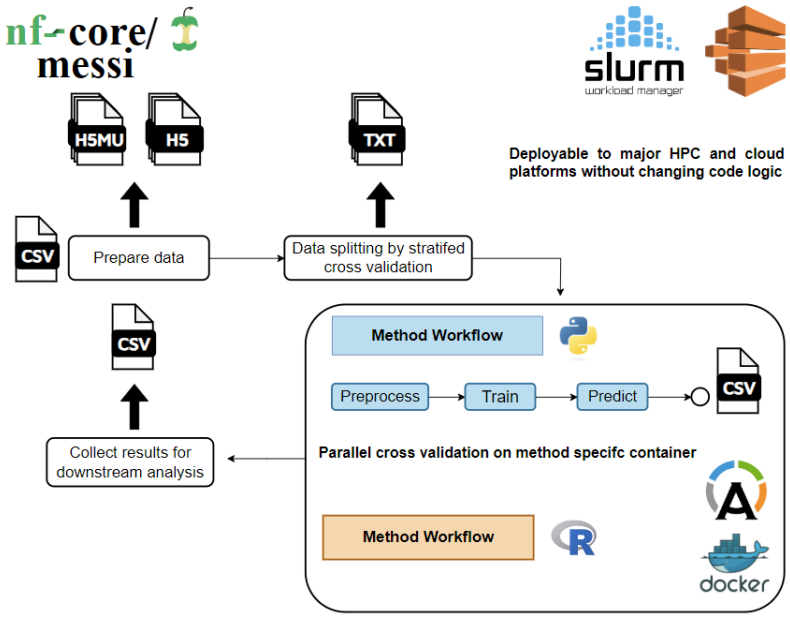

Figure 1: Workflow design of MESSI.

Benchmark analysis

We applied the pipeline on the described methods upon the real datasets. We presented benchmark results for various multiomics data and methods at Fig 2 with real datasets from table 1.

Simulation studies

We examined the methods on controlled setting of simulation studies. Methods were tested against multiomics data with varying signal, correlation, number of samples, and number of predictors on Fig **??**.

Computational time

We observed notable variations in the computation time of most methods with changing dataset sizes (categorized as small if the size of dataset is less than median of all), while DIABLO exhibited consistent performance regardless of dataset size. Conversely, Cooperative Learning displayed a comparatively longer median time than other methods. (Fig 1A).

Real data classification performance

Regarding classification performance, we employed the area under the curve (AUC) score for method comparison across datasets (Fig 1B). Mogonet emerged as the weakest performer, with MOFA + glmnet showing only marginal improvement. In contrast, Cooperative Learning demonstrated the highest performance, followed by DIABLO and RGCCA. Interestingly, DIABLO with a null design outperformed than its full design choice. Nevertheless, all methods performed poorly on the tcga-thca dataset.

Feature performance

We further investigated each method's ability to identify significant biomarkers and examined the overlap between them, calculating the similarity in their feature sets (Fig 1C). Most methods identified similar biomarkers in TCGA datasets but exhibited minimal overlap in the rosmap and gse71669 datasets. Notably, DIABLO and RGCCA tended to select similar biomarkers due to their shared backend framework of canonical correlation analysis.

Table 1: Overview of real datasets to benchmark

Dataset	N	Y=0	Y=1	Prop(Y = 1)	Omic	P	Disease
GSE38609	24	Non-autistic disorder	Autistic disorder	0.458	mrna	8110	Autistic disorder
					cpg	1674	
					cc	12	
GSE71669	33	Non-bladder cancer	Bladder cancer	0.424	mrna	5831	Bladder cancer
					cpg	8915	
					cc	10	
ROSMAP	351	Non-alzheimer	Alzheimer	0.519	epigenomics	58	Alzheimer's Disease
					genomics	74	
					transcriptomics	90	
TCGA-BLCA	336	Non-bladder cancer	Bladder cancer	0.685	Methylation_HM450K	8072	Bladder cancer
					miRNA	285	
					RPPA	44	
					RNAseq_HiSeq	6376	
TCGA-BRCA	109	Non-breast cancer	Breast cancer	0.358	Methylation_HM450K	7985	Breast cancer
					miRNA_GA	282	
					RPPA	43	
					RNAseq_HiSeq	6191	
TCGA-KIPAN	123	Non-kidney cancer	Kidney cancer	0.520	Methylation_HM450K	7914	Kidney cancer
					miRNA_GA_miR	225	
					RPPA	60	
					RNAseq_HiSeq	6145	
TCGA-THCA	217	Non-thyroid cancer	Thyroid cancer	0.318	Methylation_HM450K	6367	Thyroid cancer
					miRNA	256	
					RPPA	25	
					RNAseq_HiSeq	5692	

* Features represents number of predictors in invididual omics

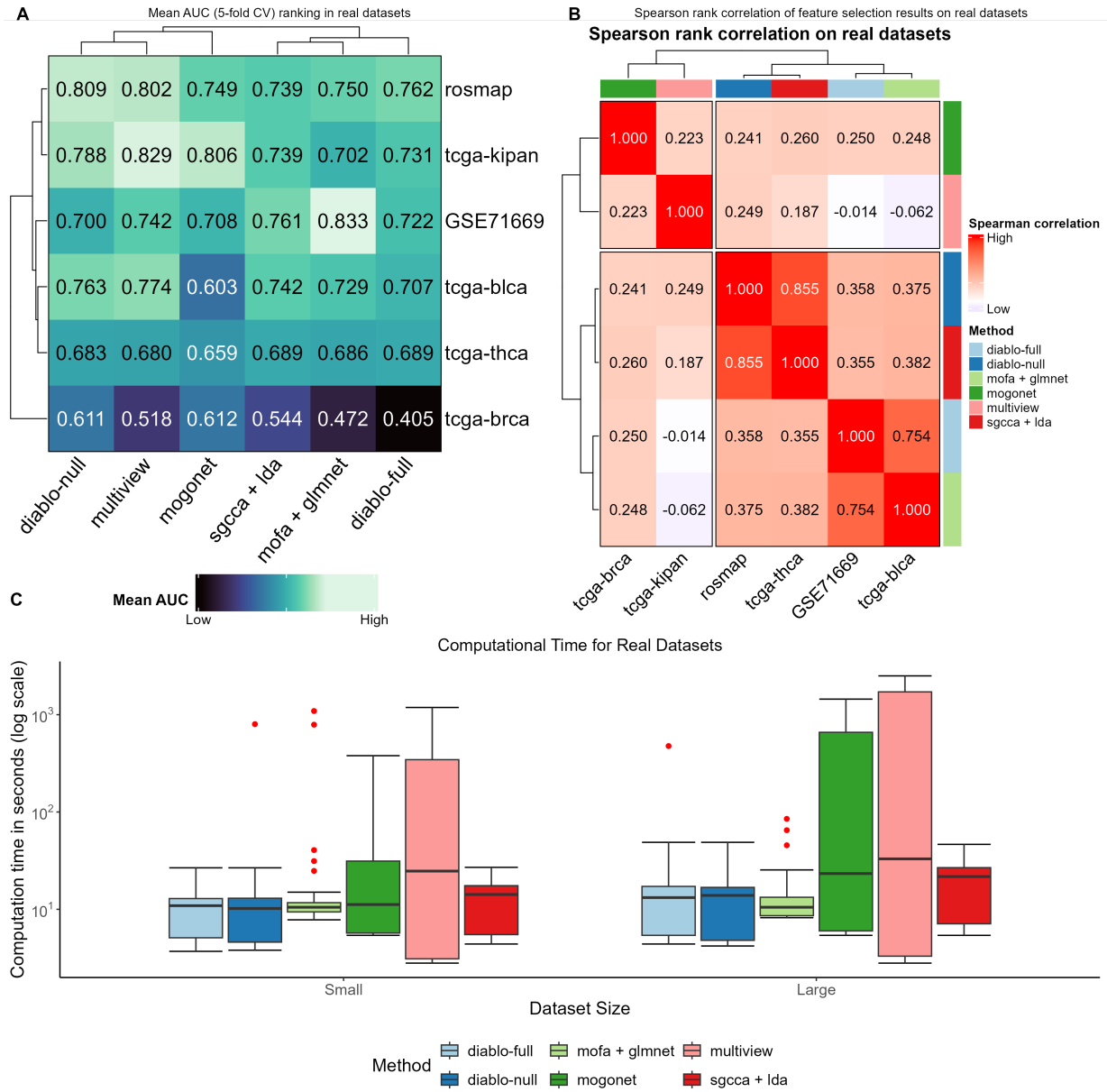

Figure 2: Benchmark results on real datasets.

Discussion

- Explain results, and relevant comments to these

Conclusion

Overall, Cooperative Learning emerged as the top performer in terms of classification performance, disregarding its computational time overhead. Conversely, MOGONET, leveraging deep neural networks, displayed the weakest performance. While acknowledging the potential for further enhancements through exploration of additional datasets and methodologies, we encountered a significant challenge of no publicly available repository/data portal with curated multiomics data. To address this gap, we have begun to curate multiomics studies for various omics data (e.g., transcriptomics, methylation, proteomics) and associated clinical metadata across a range of cancer and non-cancer diseases from various public sources to create the first data portal for multiomics data ($\sim$ 100 studies) in both standard formats mentioned such that they can easily be used by other researchers. We anticipate this exercise to be highly relevant to the community utilizing multiomics data for their research, such will help inform the development of a new integrative method that is applicable to different types (bulk, single-cell, or spatial) of multiomics data with the extensibility of our proposed MESSI pipeline.

Methods

Metric

First, defined the metrics used in benchmark analysis ...

- TP: true positive
- TN: true negative
- FP: false positive
- FN: false negative

wheras, it could be constructed to the following:

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Method Algorithms

We present here the mathematical details of the integration methods benchmarked ...

DIABLO

DIABLO is ...

Multiview

Multiview is ...

MOFA

MOFA is ...

MOGONET

MOGONET is ...

RGCCA

RGCCA is ...

TODO

- Explain what is the data input in mathematical form
- Then introduce the integration methods used here
- Each should have their unique characterisitic, and what problem it solves in math, with proper citation
- Then on how data was simulated, hopefully add math details
- Then explain what metrics to be used in both simulated data and real datasets
  - Define what is TP, FP, FN, TN
  - and the the actual formulas of the metrics

References

Argelaguet R., Arnol D., Bredikhin D., Deloro Y., Velten B., Marioni J.C. & Stegle O. (2020). MOFA+: A statistical framework for comprehensive integration of multi-modal single-cell data. Genome Biology 21 (1): 111.

Argelaguet R., Velten B., Arnol D., Dietrich S., Zenz T., Marioni J.C., Buettner F., Huber W. & Stegle O. (2018). Multi-omics factor analysis-a framework for unsupervised integration of multi-omics data sets. Mol Syst Biol 14 (6): e8124.

Bersanelli M., Mosca E., Remondini D., Giampieri E., Sala C., Castellani G. & Milanesi L. (2016). Methods for the integration of multi-omics data: Mathematical aspects. BMC bioinformatics 17: 167–177.

Bredikhin D., Kats I. & Stegle O. (2022). MUON: Multimodal omics analysis framework. Genome Biology 23 (1): 42.

Cantini L., Zakeri P., Hernandez C., Naldi A., Thieffry D., Remy E. & Baudot A. (2021). Benchmarking joint multi-omics dimensionality reduction approaches for the study of cancer. Nature communications 12 (1): 124.

Di Tommaso P., Chatzou M., Floden E.W., Barja P.P., Palumbo E. & Notredame C. (2017). Nextflow enables reproducible computational workflows. Nature biotechnology 35 (4): 316–319.

Ding D.Y., Li S., Narasimhan B. & Tibshirani R. (2022). Cooperative learning for multiview analysis. Proceedings of the National Academy of Sciences 119 (38): e2202113119.

Docker I. & others (2020). Docker. lınea].[Junio de 2017]. Disponible en: https://www. docker. com/what-docker.

Gentleman R.C., Carey V.J., Bates D.M., Bolstad B., Dettling M., Dudoit S., Ellis B., Gautier L., Ge Y., Gentry J. & others (2004). Bioconductor: Open software development for computational biology and bioinformatics. Genome biology 5: 1–16.

Girka F., Camenen E., Peltier C., Gloaguen A., Guillemot V., Le Brusquet L. & Tenenhaus A. (2023). Multiblock data analysis with the RGCCA package. Journal of Statistical Software 1–36.

Hasin Y., Seldin M. & Lusis A. (2017). Multi-omics approaches to disease. Genome biology 18 (1): 1–15.

Huang S., Chaudhary K. & Garmire L.X. (2017). More is better: Recent progress in multi-omics data integration methods. Frontiers in genetics 8: 268903.

Köster J. & Rahmann S. (2012). Snakemake—a scalable bioinformatics workflow engine. Bioinformatics 28 (19): 2520–2522.

Kurtzer G.M., Sochat V. & Bauer M.W. (2017). Singularity: Scientific containers for mobility of compute. PloS one 12 (5): e0177459.

Li Y., Mansmann U., Du S. & Hornung R. (2022). Benchmark study of feature selection strategies for multi-omics data. BMC bioinformatics 23 (1): 1–18.

Li Y., Wu F.-X. & Ngom A. (2018). A review on machine learning principles for multi-view biological data integration. Briefings in bioinformatics 19 (2): 325–340.

Luecken M.D., Büttner M., Chaichoompu K., Danese A., Interlandi M., Müller M.F., Strobl D.C., Zappia L., Dugas M., Colomé-Tatché M. & others (2022). Benchmarking atlas-level data integration in single-cell genomics. Nature methods 19 (1): 41–50.

Pucher B.M., Zeleznik O.A. & Thallinger G.G. (2019). Comparison and evaluation of integrative methods for the analysis of multilevel omics data: A study based on simulated and experimental cancer data. Briefings in bioinformatics 20 (2): 671–681.

R Core Team R. & others (2013). R: A language and environment for statistical computing.

Ramos M., Schiffer L., Re A., Azhar R., Basunia A., Rodriguez C., Chan T., Chapman P., Davis S.R., Gomez-Cabrero D. & others (2017). Software for the integration of multiomics experiments in bioconductor. Cancer research 77 (21): e39–e42.

Richardson S., Tseng G.C. & Sun W. (2016). Statistical methods in integrative genomics. Annual review of statistics and its application 3: 181–209.

Sanner M.F. & others (1999). Python: A programming language for software integration and development. J Mol Graph Model 17 (1): 57–61.

Singh A., Shannon C.P., Gautier B., Rohart F., Vacher M., Tebbutt S.J. & Lê Cao K.-A. (2019). DIABLO: An integrative approach for identifying key molecular drivers from multi-omics assays. Bioinformatics 35 (17): 3055–3062.

Stallman R.M., McGrath R. & Smith P. (1988). GNU make. Free Software Foundation, Boston.

Subramanian I., Verma S., Kumar S., Jere A. & Anamika K. (2020). Multi-omics data integration, interpretation, and its application. Bioinformatics and biology insights 14: 1177932219899051.

Sun Y.V. & Hu Y.-J. (2016). Integrative analysis of multi-omics data for discovery and functional studies of complex human diseases. Advances in genetics 93: 147–190.

Vandereyken K., Sifrim A., Thienpont B. & Voet T. (2023). Methods and applications for single-cell and spatial

multi-omics. Nature Reviews Genetics 24 (8): 494–515.
Wang T., Shao W., Huang Z., Tang H., Zhang J., Ding Z. & Huang K. (2021). MOGONET integrates multi-omics data using graph convolutional networks allowing patient classification and biomarker identification. Nature Communications 12 (1): 3445.
Yu X.-T. & Zeng T. (2018). Integrative analysis of omics big data. Computational Systems Biology: Methods and Protocols 109–135.
Zeng I.S.L. & Lumley T. (2018). Review of statistical learning methods in integrated omics studies (an integrated information science). Bioinformatics and biology insights 12: 1177932218759292.

More information

This will be Appendix A.

One more thing

This will be Appendix B.